UNIVERSITY OF BIRMINGHAM

Functional Programming
Week 6 – Consolidation / Revision

Eric Finster
Martin Escardo
Mian M. Hamayun

Module Outline – What we have covered so far?

- Week 1
  - Introduction to Haskell - Conway's Game of Life.
  - Types in Haskell
  - Polymorphism
- Week 2
  - Type classes and instances
  - Functions
  - More on type classes
- Week 3
  - Even more on type classes
  - List comprehensions
- Week 4
  - Recursive functions
  - Higher-order functions

Module Outline (cont'd)

- Week 5
  - User defined data types I
- Week 6
  - Consolidation Week
- Week 7
  - Lazy natural numbers
  - User defined data types II
- Week 8 & 9
  - Monads
- Week 10
  - Implementing a small imperative language
- Week 11
  - Implementing a small imperative language
  - Rigorous programming

- Additional, optional materials
  - A model IO' of the IO monad
  - Unbeatable tic-tac-toe
  - Elements of Laziness
- Module Outline on Gitlab

Common Issues / Technical Problems

- No Git Repository in Jupyterhub work directory
  - No SSH Setup done till date!
- Working with ‘old version’ of the Git Repository (git pull)
- Difficulty with running ‘presubmit.sh’ script
  - Some times getting permissions error (chmod +x ./presubmit.sh)
  - Copy/pasting file contents from browser to Jupyterhub!
  - Using incorrect name for the haskell file (PracticeTest.hs)

Common Issues / Technical Problems (2)

- Compilation errors – especially towards the end of test
  - Incremental development / submission
  - Using ‘undefined’ definitions to avoid errors
  - Commenting-out non-compiling code
- Working in a incorrect file/directory
  - Missing ‘Types.hs’ and other required files.
- Submission of incorrect code file!

Practice Test: Question 1 – Parity

We say that a byte (a sequence of 8 bits) has even parity if the number of 1s is even.

Implementation Task

Write a function

```
checkParity :: String -> Bool
checkParity = undefined
```

which takes as input a string of bits and checks that

1. the string size is a multiple of 8, and
2. each byte in the string has even parity.

The function should return True if both conditions are met, and False otherwise. We are representing bits here by the characters 0 and 1. You may assume that the input strings contain only 0s and 1s.

Practice Test: Question 2 – Substitution

A *substitution cipher* is an old method of encryption, in which the cipher takes a string and a *key* that is as long as the alphabet that the message uses. In our case, the message will be expressed using the English alphabet so our cipher key will be a string of length 26. This represents a mapping of each letter of the alphabet to a different letter.

For example, the key "LYKBDOCAWITNVRHJXPUMZSGEQF" maps 'A' to 'L', 'B' to 'Y', 'C' to 'K' and so on.

Implementation Task

Write a function

```
substitution :: String -> String -> String
substitution plaintext key = undefined
```

which takes a plaintext string (that might contain punctuation and spaces) and an uppercase key and returns the ciphertext.

Practice Test: Question 3 – Primes (Part I)

A famous theorem about prime numbers (called *Chebyshev's Theorem*) asserts that for any number n, there always exists a prime number p such that $n < p < 2n$. That is, there is always a prime number between n and 2n.

Implementation Task

Write a function

```
largestPrimeBetween :: Int -> Int
largestPrimeBetween = undefined
```

which returns the largest prime between n and 2n.

Practice Test: Question 3 – Primes (Part II)

In number theory, a strong prime is a prime number that is greater than the average of the nearest prime above and below. In other words, it is closer to the succeeding prime than it is to the preceding one.

For example, 17 is the seventh prime: the sixth and eighth primes, 13 and 19, add up to 32, and half of that is 16; 17 is greater than 16, so 17 is a strong prime.

Implementation Task

Write a function

```
strongPrimes :: Int -> [Int]
strongPrimes n = undefined
```

which takes as input the integer n and prints the first n strong prime numbers.

Practice Test: Question 4 – Directions

Consider the following encoding of directions using the type Int

- 0 encodes a movement to the left
- 1 encodes a movement to the right
- 2 encodes a movement upwards
- 3 encodes a movement downwards
- other values of type Int encode no movement

For readability, we introduce the following type alias

```
type Direction = Int
```

Let us define the type Command to consist of a pair of a Direction and an Int.

```
type Command = (Direction, Int)
```

Practice Test: Question 4 – Directions (2)

Given a coordinate pair (x, y), the execution of a command consists in incrementing the corresponding coordinate.

So for example, executing (0, 10) on the pair (5, 5) should result in (-5, 5). (We use the mathematical indexing: "right" means increasing the x coordinate and "up" means increasing the y coordinate).

Implementation Task

Write a function which, given an initial position (x, y), computes the final position after the execution of a list of commands.

```
executeCommands :: [Command] -> (Int, Int) -> (Int, Int)
executeCommands = undefined
```

Practice Test: Question 5 – Babylonian Palindromes

We say a number is a palindrome if it has at least two digits appears the same when its digits are reversed. For example 14341 is a palindrome, while 145 is not.

The notion of being a palidrome, however, is not intrinsic to a number since it depends on which base we use to express it (the examples above are given in base 10). For example, the number 21 is not a palindrome in base 10, while its representation in binary (i.e., base 2) is 10101 which is a palindrome.

Different cultures have used different bases for their number systems throughout history. The Babylonians, for example, wrote numbers in base 60.

Implementation Task

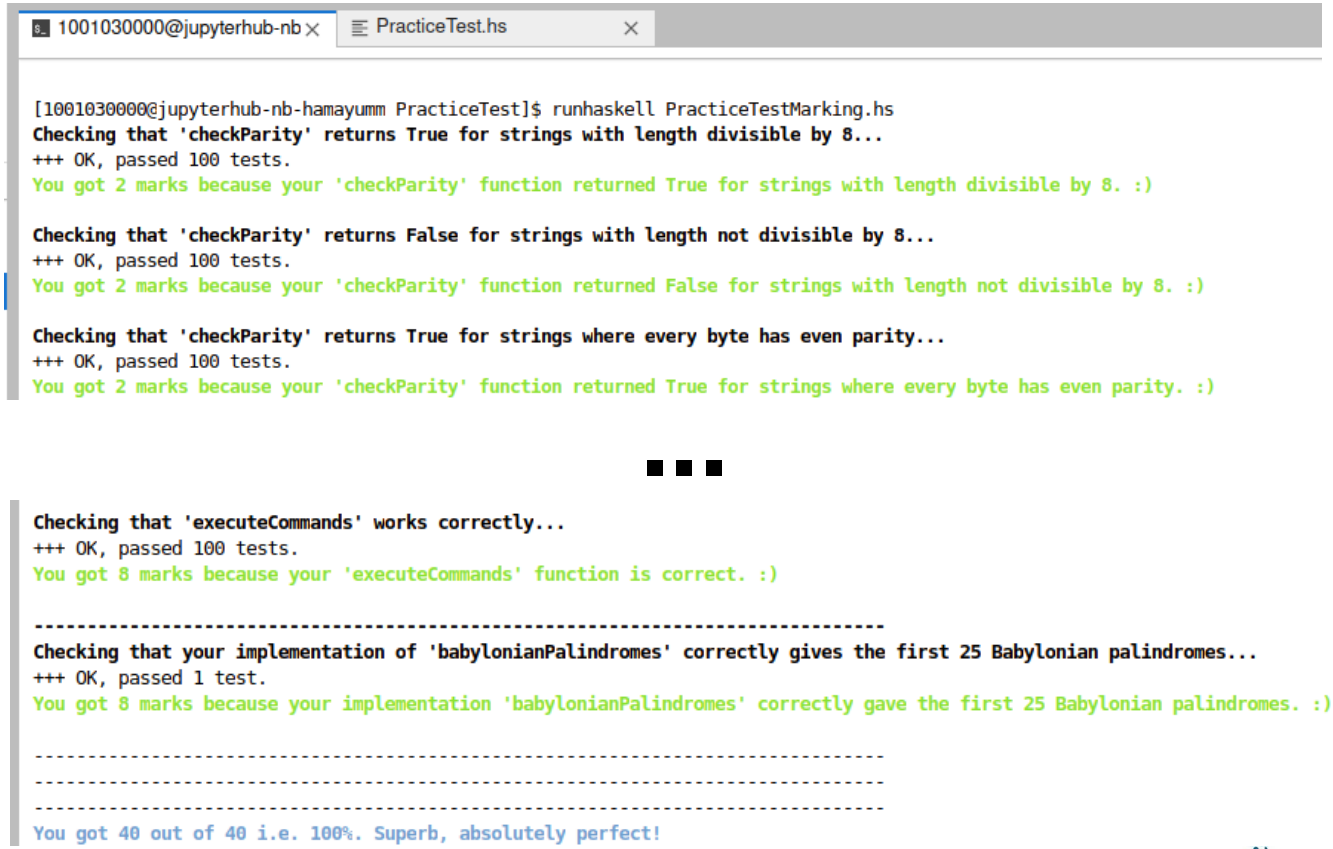
Write a function

```
babylonianPalindromes :: [Integer]
babylonianPalindromes = undefined
```

which produces the infinite list of Babylonian palindromes.

Marking Your Submission & Getting Feedback

- You can use the ‘runhaskell PracticeTestMarking.hs’ command to mark your submission and see the feedback.



Change in Classtest 1 & 2 Timings – Please Note!

- The timings for the Class Test 1 & 2 have been changed!
- Updated timings are:
  - Class Test 1: Thursday, 09 Nov, 2023, **3pm-5pm** (UAE)
  - Class Test 2: Thursday, 07 Dec, 2023, **3pm-5pm** (UAE)
- Venue for both tests will be **E-Lab 1326**
- Please reach the venue 15 minutes before the start!
- Seating Plan will be available outside the venue.
- Personal Devices (Mobile, Tablets, Laptops) will not be allowed.

Additional Resources for Preparation

Some of the online resources that you can use/consult:

- H-99: Ninety-Nine Haskell Problems
  - http://wiki.haskell.org/H-99:_Ninety-Nine_Haskell_Problems
- Codewars
  - https://www.codewars.com/collections/
- Project Euler
  - https://projecteuler.net/archives
- Exercism
  - https://exercism.org/tracks/haskell
- Codingame
  - https://www.codingame.com/start/
- 15 Resources to Help You Learn Haskell in 2023
  - https://serokell.io/blog/how-to-learn-haskell-in-10-minutes